C# / .NET Learning Roadmap

Topic Notes — Section 3: Generics

Topics: 3.1 through 3.2

Date: 29 June 2026

Section 3 — Generics

This section covers C# generics — the mechanism that enables type-safe, reusable code across any type. Both topics build directly on classes, interfaces, and abstract classes from Section 2.

3.1. Generic Classes and Methods

Builds on: 2.1 (classes — generics are a modifier on class definitions), 2.9 (interfaces — generic interfaces like IEnumerable and IRepository are everywhere), 1.4 (types — generics are about working with types as parameters).

What are generics and why do they exist?

Without generics, a class that works with different types requires either duplicate classes for each type (brittle) or using object to accept anything (unsafe). Generics solve both — one implementation, type-safe for any type.

```
// Option 2 -- using object (unsafe):
public class ObjectList
{
    private object[] _items;
    public void Add(object item) { _items[_count++] = item; }
    public object Get(int index) => _items[index];
}

var list = new ObjectList();
list.Add(42); list.Add("hello");   // no compile error -- mixing types
int bad = (int)list.Get(1);        // InvalidCastException at runtime

// With generics -- one implementation, type-safe:
public class TypedList<T>
{
    private T[] _items = new T[16];
    private int _count;
    public void Add(T item) { _items[_count++] = item; }
    public T Get(int index) => _items[index];
}

var intList = new TypedList<int>();
intList.Add(42);
intList.Add("hello");   // compile error -- type-safe

string s = new TypedList<string>().Get(0);   // no cast needed
```

Generic classes

```
public class Repository<T> where T : class
{
    private readonly List<T> _store = new();

    public void Add(T item) => _store.Add(item);
    public T? GetById(int index) => index < _store.Count ? _store[index] : null;
    public IEnumerable<T> GetAll() => _store.AsReadOnly();
```

```
    public int Count => _store.Count;
}

var orderRepo = new Repository<Order>();
Order? o = orderRepo.GetById(0);   // returns Order, not object

var userRepo = new Repository<User>();
User? u = userRepo.GetById(0);    // returns User
```

Multiple type parameters

```
public class Pair<TFirst, TSecond>
{
    public TFirst First { get; }
    public TSecond Second { get; }
    public Pair(TFirst first, TSecond second) { First = first; Second = second; }
    public void Deconstruct(out TFirst first, out TSecond second)
        => (first, second) = (First, Second);
}

var pair = new Pair<string, int>("Sathish", 42);
var (name, age) = pair;   // deconstruction works
```

Generic methods

```
public class Utilities
{
    public static T? FirstOrDefault<T>(IEnumerable<T> source, Func<T, bool> predicate)
    {
        foreach (var item in source)
            if (predicate(item)) return item;
        return default;
    }

    public static void Swap<T>(ref T a, ref T b)
        => (a, b) = (b, a);

    public static List<T> CreateList<T>(params T[] items) => new List<T>(items);
}

// Type is inferred -- no need to write <Order>:
var order = Utilities.FirstOrDefault(orders, o => o.Amount > 100);

int x = 1, y = 2;
Utilities.Swap(ref x, ref y);
Console.WriteLine($"{x}, {y}");   // "2, 1"
```

Generic interfaces

```
IEnumerable<T>             // iteration
ICollection<T>             // Count, Add, Remove, Contains
IList<T>                   // indexer, IndexOf, Insert
IDictionary<TKey, TValue>  // key-value pairs
IQueryable<T>              // EF Core LINQ queries
```

The default keyword with generics

```
public T? GetFirstOrDefault<T>(IEnumerable<T> source)
{
    foreach (var item in source) return item;
    return default;   // null for reference types, 0 for int, etc.
}
```

PagedResult and Result patterns

```
public class PagedResult<T>
{
    public IReadOnlyList<T> Items { get; }
    public int TotalCount { get; }
    public int Page { get; }
    public int PageSize { get; }
    public int TotalPages => (int)Math.Ceiling(TotalCount / (double)PageSize);

    public PagedResult(IEnumerable<T> items, int totalCount, int page, int pageSize)
    { Items = items.ToList().AsReadOnly(); TotalCount = totalCount; Page = page; PageSize = pageS
}

public class Result<T>
{
    public bool IsSuccess { get; }
    public T? Value { get; }
    public string? Error { get; }

    private Result(bool ok, T? v, string? e) { IsSuccess = ok; Value = v; Error = e; }
    public static Result<T> Success(T value) => new(true, value, null);
    public static Result<T> Failure(string error) => new(false, default, error);
}
```

Key Takeaways

- Generics let you write one type-safe implementation that works for any type
- T is a type parameter — a placeholder replaced by a real type at use time
- The compiler generates optimised code per concrete type — no boxing for value types
- Generic methods infer T from arguments — you rarely need to specify it explicitly
- default(T) returns the appropriate zero/null for whatever T is

```
Syntax                | Meaning
----------------------+-----------------------------------------
class Foo<T>          | Generic class with one type parameter
class Foo<T, U>       | Generic class with two type parameters
void Method<T>(T arg) | Generic method
new Foo<int>()        | Instantiate with concrete type
default(T) or default | Zero/null for type T
```

.NET-specific rules:

- List, Dictionary, IEnumerable — BCL is built on generics
- PagedResult and Result are patterns used in almost every .NET API
- IRepository — the standard generic repository pattern in Clean Architecture

Memory aid: Generics are like cookie cutters. The cutter (generic class) is the same shape every time. The cookie depends on what dough (type) you push through it.

Debugging Tips

Symptom	Cause	Fix
The type 'X' cannot be used as type parameter 'T'	Type constraint not satisfied	Check where constraints — covered in 3.2
Cannot call .ToString() on T	Compiler doesn't know T has that method	Add where T : class or a more specific constraint
default returns 0 instead of null for T	T is a value type — default(int) is 0	Use T? return type or add where T : class
Generic method type argument not inferred	Ambiguous or no argument to infer from	Specify type explicitly: Method()
Value type gets boxed in generic	Using object instead of T somewhere	Ensure all operations stay in T — don't cast to object

.NET-specific: List vs ArrayList — ArrayList stores object. Never use it in new code. Always use List — type-safe, no boxing for value types.

Self-check — you should now be able to:

- Write a generic class with one and two type parameters
- Explain why generics are better than using object as a universal type
- Write a generic method and show that the compiler infers T from the argument
- Explain what default(T) returns for reference types vs value types

3.2. Generic Constraints (where)

Builds on: 3.1 (generic classes and methods — constraints restrict the type parameters you defined there), 2.9 (interfaces — interface constraints are the most common kind), 2.8 (abstract classes — class constraints can require a base class).

What are constraints and why do they exist?

In 3.1, T was completely unconstrained — the compiler can't guarantee T has any particular method or property. Constraints tell the compiler 'T must satisfy these requirements,' unlocking the ability to call methods and access properties.

```
// Without constraint -- compile error:
public class Repository<T>
{
    public T? FindById(int id)
        => _items.FirstOrDefault(i => i.Id == id);   // compile error -- T has no Id
}

// With constraint -- Id is guaranteed:
public interface IEntity { int Id { get; } }
```

```
public class Repository<T> where T : IEntity
{
    public T? FindById(int id)
        => _items.FirstOrDefault(i => i.Id == id);   // works
}
```

Interface constraint: where T : SomeInterface

```
public class Sorter<T> where T : IComparable<T>
{
    public T[] Sort(T[] items) { Array.Sort(items); return items; }
}

var sorter = new Sorter<int>();      // int implements IComparable<int>
var sorter2 = new Sorter<string>();  // string implements IComparable<string>
```

where T : class, where T : struct, where T : new()

```
// class -- T must be a reference type:
public class Cache<T> where T : class
{
    private readonly Dictionary<string, T> _store = new();
    public T? Get(string key) => _store.TryGetValue(key, out var v) ? v : null;
    public void Set(string key, T value) => _store[key] = value;
}

// struct -- T must be a value type:
public class NullableWrapper<T> where T : struct
{
    private T? _value;
    public void Set(T value) => _value = value;
    public T? Get() => _value;
}

// new() -- T must have a public parameterless constructor:
public class Factory<T> where T : new()
{
    public T Create() => new T();   // valid because of new() constraint
}
```

Base class and notnull constraints

```
// Base class constraint:
public abstract class BaseEntity
{ public int Id { get; set; } public DateTime CreatedAt { get; set; } = DateTime.UtcNow; }

public class GenericService<T> where T : BaseEntity
{
    public async Task<List<T>> GetRecentAsync(int days = 30)
        => await _context.Set<T>()
            .Where(e => e.CreatedAt >= DateTime.UtcNow.AddDays(-days))
            .ToListAsync();
            // e.CreatedAt is available because T : BaseEntity
}
// notnull -- T cannot be a nullable reference type:
```

```
public class Registry<T> where T : notnull
{
    private readonly Dictionary<T, string> _map = new();
    public void Register(T key, string value) => _map[key] = value;
}
```

Combining multiple constraints

```
public class AuditableService<T>
    where T : BaseEntity, IValidatable, new()
{
    public T CreateAndValidate()
    {
        var entity = new T();               // new() constraint
        entity.CreatedAt = DateTime.UtcNow; // BaseEntity constraint
        entity.Validate();                  // IValidatable constraint
        return entity;
    }
}

// Correct order: class or struct first, new() always last
where T : class, IEntity, IValidatable, new()

// Multiple type parameters with independent constraints:
public class Converter<TInput, TOutput>
    where TInput : class
    where TOutput : class, new() { ... }
```

EF Core generic repository

```
public interface IRepository<T> where T : class
{
    Task<T?> GetByIdAsync(int id);
    Task<IEnumerable<T>> GetAllAsync();
    Task AddAsync(T entity);
    Task UpdateAsync(T entity);
    Task DeleteAsync(T entity);
}

public class EfRepository<T> : IRepository<T> where T : class
{
    protected readonly AppDbContext _context;
    protected readonly DbSet<T> _dbSet;

    public EfRepository(AppDbContext context)
        => (_context, _dbSet) = (context, context.Set<T>());

    public async Task<T?> GetByIdAsync(int id) => await _dbSet.FindAsync(id);
    public async Task<IEnumerable<T>> GetAllAsync() => await _dbSet.ToListAsync();
    public async Task AddAsync(T entity) { await _dbSet.AddAsync(entity); await _context.SaveChan
    public async Task UpdateAsync(T entity) { _dbSet.Update(entity); await _context.SaveChangesAs
    public async Task DeleteAsync(T entity) { _dbSet.Remove(entity); await _context.SaveChangesAs
}

// Open generic DI registration:
```

```
builder.Services.AddScoped(typeof(IRepository<>), typeof(EfRepository<>));
```

Key Takeaways

- Constraints restrict what types can be used as T — unlocking capabilities beyond what all types share
- where T : interface — most common, guarantees specific methods/properties exist
- where T : class — T is a reference type, null is valid
- where T : struct — T is a value type, never null
- where T : new() — T has a parameterless constructor, new T() is valid
- where T : BaseClass — T inherits from a specific class, its members are available
- class or struct first, new() always last in constraint list

```
Constraint        | Syntax             | What it guarantees
------------------+--------------------+---------------------------------------
Interface         | where T : IFoo     | T implements IFoo
Reference type    | where T : class    | T is a class/interface/delegate/array
Value type        | where T : struct   | T is a struct/enum, never null
Constructor       | where T : new()    | T has public parameterless constructor
Base class        | where T : Foo      | T inherits from Foo
Not null          | where T : notnull  | T cannot be nullable reference type
Combined          | class, IFoo, new() | All must be satisfied
```

.NET-specific rules:

- EF Core's DbSet requires where T : class — entities must be reference types
- AddScoped(typeof(IRepository<>), typeof(EfRepository<>)) — open generic DI registration
- IComparable constraint enables sorting; IEquatable enables equality comparison

Memory aid: Constraints are like job requirements on a hiring poster. where T : IEntity means 'must implement IEntity' — just as a job posting says 'must have X qualification.'

Debugging Tips

Symptom	Cause	Fix
T doesn't contain definition for X	No constraint guaranteeing that member exists on T	Add interface or class constraint that has the member
new T() compile error	No new() constraint	Add where T : new()
Cannot use null with T	T might be a value type	Add where T : class or use T?
struct and class constraints together	Mutually exclusive — compile error	Use one or the other, not both
new() not last in constraint list	Wrong constraint order — compile error	Move new() to end: where T : class, IFoo, new()
Open generic DI registration not working	Wrong typeof syntax	Use typeof(IRepo<>) not typeof(IRepo)

.NET-specific: When using EF Core with a generic repository, the where T : class constraint is required because DbSet itself has that constraint.

Self-check — you should now be able to:

- Explain why an unconstrained T limits what you can do with it
- Write a generic class with an interface constraint and access the interface members on T
- Choose the correct constraint for each scenario: class, struct, new(), interface, base class
- Combine multiple constraints correctly in the right order
- Use open generic DI registration with typeof(IRepository<>)

Section 3 complete. Both topics 3.1 and 3.2 are covered. Say next to begin Section 4: Collections.